

# Set 3

---

Below you can find the instructions for each exercise in **Set 3**.

You can create a workspace for your **Set3** folder in **VS Code** by selecting **Open Folder...** and choosing your **Set3** folder. This allows you to access and manage all your exercise files in one place.

Open the **.c** file you want to run using **VS Code** or any other text editor you prefer (for example, Notepad on Windows). You can edit your C file directly in any of these editors.

When you finish writing your code, you need to **compile it**. To do this, open **w64devkit** and navigate to the **exercise folder** you are working on within your **Set3** directory. For detailed steps on how to navigate to the folder and compile your program, refer to the *"Getting started with LEU483"* guide on Canvas.

You don't need to do all exercises at once, some can be saved to practice for the exam. However, it is recommended that you at least finish exercises 1-5 before starting to work on set 4.

## 1 Pointer basics

Open: `ex_ptr_basics/main.c`

Samples: `sample_ptr`

In this exercise you will look at the addresses and values of an integer variable and an integer pointer. Always use exactly the variable that is mentioned in each task.

### Task:

- The pointer `ptr` shall point to `a`.
- Print the value of `a` and the value that `ptr` points to.
- Print the address of `a` and the value of `ptr`.
- Print the address of `ptr`.

## 2 Integer pointer

Open: `ex_int_ptr/main.c`

Samples: `sample_ptr`, `sample_ptr_func`

In this program you will change the value of a variable of type `int` using pointers.

### Task:

- Use the pointer `ptr` to change the value of `a` to be 15 instead of 6.
- Implement a function called `modify_int` that modifies the value of an integer. This should be a `void` function, i.e., the function should not return any value.
- Use the function `modify_int` to change the value of `a` to be 23 instead.

### Program output:

```
Original value of a: 6  
New value of a: 15  
Final value of a: 23
```

## 3 Sort integers

Open: `ex_sort_int/main.c`

Samples: `sample_ptr`, `sample_ptr_func`

This program asks the user for two integers and sorts them.

### Task:

- Implement a function called `swap` that swaps the values of two integers. The function should be a `void` function, i.e., the function should not return any value.
- Implement another function called `sort_int` that sorts two integers. This function should also be a `void` function and `swap` should be used within this function.
- Use `sort_int` to sort the values provided by the user such that the smaller value is placed in `num1` and the larger value is placed in `num2`.

### Example output:

```
Enter two integers:      (first run)  
first integer > 64  
second integer > 4  
num1 = 4, num2 = 64  
  
Enter two integers:      (second run)  
first integer > 56  
second integer > 78  
num1 = 56, num2 = 78
```

## 4 Alphabet

Open: `ex_alphabet/main.c`

### Task:

- Use only `for` loops (nested), `printf` and one variable of type `char` to print the alphabet in two rows. It is okay to use integer indices in the loops.

### Program output:

```
a b c d e f g h i j k l m
n o p q r s t u v w x y z
```

## 5 Christmas tree

Open: `ex_xmas_tree/main.c`

Decorate the branches of a christmas tree in terminal, using `getchar` and `putchar`.

### Task:

- The program should ask the user for input to pick a christmas tree decoration. You should use `getchar` to do this.
- The program should then print the decorated tree, where the decoration is shown on the top of the tree and the ends of the branches. You should use `putchar` to do this.

### Notes:

- A function to print an undecorated christmas tree is provided in the code. You need to edit the function so that it prints out a decorated christmas tree. Try and understand what the code is doing first, and then implement `putchar` when you make your changes.
- It is enough to replace parts of the tree with the decoration, i.e., you can keep `tree_height` as it is.

### Program output:

```
Enter a decoration character: o
  o
 o*o
o***o
o*****o
o*****o
  |
  |
```

## 6 Shift integers

Open: `ex_shift/main.c`

Samples: `sample_ptr_func`

In this exercise you will implement two (quite useless) functions that shift the values of 3 integers either forwards or backwards using pointers. The functions take pointers to `num1`, `num2` and `num3` as arguments and shift the values one step.

### Task:

- Implement the function `shift_forward`. This function should shift the values of the arguments as `num1 = num2, num2 = num3` and `num3 = num1`.
- Implement the function `shift_backward`. This function should shift the values of the arguments as `num1 = num3, num2 = num1, num3 = num2`.
- Check that you get back the original values of `num1`, `num2` and `num3` after calling the functions once each.
- `shift_backward` is unnecessary since the same effect can be achieved by calling `shift_forward`. Use `shift_forward` instead of `shift_backward` to get the original values of `num1`, `num2` and `num3`. Note that you should only call `shift_forward` once to get the values back.

### Program output:

```
Before shift:
num1 = 3, num2 = 11, num3 = 23
After forward shift:
num1 = 11, num2 = 23, num3 = 3
After backward shift:
num1 = 3, num2 = 11, num3 = 23
```

## 7 Dice pointers

Open: ex\_dice\_ptrs/main.c

Samples: sample\_ptr\_func

In this exercise, you will solve exercise 6 Dice from set1 by using pointers instead.

### Task:

- Implement the function `five_dice` that simulates the outcome of throwing five dice. The results of the throws shall be assigned to the five dice through pointers, the function should not return any value (i.e., be a `void` function).

### Example output:

```
(first run)
Dice at beginning: 0 0 0 0 0
Dice after calling function: 1 2 4 1 1

(second run)
Dice at beginning: 0 0 0 0 0
Dice after calling function: 4 2 1 6 2
```

## 8 Recommendations

Open: `ex_switch/main.c`

### Task:

Write a small program that recommends the user where they can go to watch, buy or rent the following categories:

- Movies
- Books
- Music

The program should ask the user for input to pick one of the categories and then print some recommendations.

**Note:** For this you could use a combination of `if`, `if else` and `else`, but **for this example you should use a `switch` statement.**

### Program output:

```
Where can I find the following:
```

```
Movies (1)
```

```
Books  (2)
```

```
Music  (3)
```

```
>2
```

```
Library or book store.
```

```
Where can I find the following:
```

```
Movies (1)
```

```
Books  (2)
```

```
Music  (3)
```

```
>1
```

```
Cinema, Netflix, etc.
```

```
Where can I find the following:
```

```
Movies (1)
```

```
Books  (2)
```

```
Music  (3)
```

```
>4
```

```
4 does not correspond to a category in our list.
```

## 9 Tricky pointers

Open: `ex_tricky_ptrs/main.c`

Samples: `sample_ptr`

This is a theoretical exercise where you will think about how pointers behave.

### Task:

- Uncomment a)-i) one at a time and try to explain:
  - If compile error: What's the problem
  - If no compile error: Run it. Any problems? If so what happens? Why?  
**Note:** if exit code is not 0 there's a problem.
  - If no problem what will be printed? Why?